

Python Fundamentals

1. What is Python?

Python is a **high-level, interpreted, general-purpose, and dynamically typed programming language**.

Python is widely used in:

- Artificial Intelligence and Machine Learning
- Data Analysis
- Automation
- Web Development

Features of Python

- Easy to learn
- Readable syntax
- Interpreted
- Dynamically typed
- Object-oriented
- Large standard library
- Open-source
- Cross-platform

Example

```
print("Hello, Python")
```

2. Variables

A **variable** is a name used to store a value.

```
name = "Arati"
```

```
age = 20
```

```
marks = 85.5
```

Python does not require explicit declaration of the variable's data type.

```
x = 10
```

```
x = "Hello"
```

Here, the value and data type of x can change.

Variable Naming Rules

- A variable name cannot start with a number.
- Spaces are not allowed.
- Hyphens are not used.
- Use meaningful variable names.

```
student_name = "Arati"
```

```
age2 = 20
```

```
_marks = 90
```

3. Data Types

A data type tells Python what kind of data a variable stores.

int

Used for whole numbers.

```
age = 20
```

```
marks = 95
```

float

Used for decimal numbers.

```
cgpa = 9.0
```

```
price = 99.5
```

str

Used for text or strings.

```
name = "Arati"
```

bool

Stores either True or False.

```
is_student = True
```

```
is_passed = False
```

list

An ordered and mutable collection.

```
marks = [80, 90, 95]
```

tuple

An ordered and immutable collection.

```
point = (10, 20)
```

set

A collection of unique values.

```
numbers = {1, 2, 3}
```

dict

Stores data in key-value pairs.

```
student = {  
    "name": "Arati",  
    "age": 20  
}
```

4. Input and Output

print()

The print() function is used to display output.

```
print("Hello")
```

```
print(10)
```

input()

The input() function is used to take input from the user.

```
name = input("Enter your name: ")
```

```
print(name)
```

Important

input() normally returns a **string**.

```
age = input("Enter age: ")
```

If the user enters 20, Python receives it as "20".

To convert it into an integer:

```
age = int(input("Enter age: "))
```

5. Type Casting

Type casting means converting a value from one data type to another.

```
int("10")    # 10
```

```
float("10")  # 10.0
```

```
str(10)      # "10"
```

```
bool(1)      # True
```

Example:

```
x = "10"
```

```
y = int(x)
```

```
print(y)
```

Output:

```
10
```

Important

"10" → String

10 → Integer

6. Operators

Operators are symbols used to perform operations on values and variables.

Arithmetic Operators

+ Addition

- Subtraction

* Multiplication

/ Division

// Floor Division

% Remainder

** Power

Example:

a = 10

b = 3

a + b → 13

a - b → 7

a * b → 30

a / b → 3.333...

a // b → 3

a % b → 1

a ** b → 1000

Important

% gives the **remainder**.

10 % 3

Output:

1

** is used for **power**.

2 ** 3

Output:

8

Comparison Operators

Comparison operators compare two values and return **True or False**.

> Greater than

< Less than

>= Greater than or equal to

`<=` Less than or equal to

`==` Equal to

`!=` Not equal to

Example:

`a = 10`

`b = 5`

`print(a > b) # True`

`print(a < b) # False`

`print(a == b) # False`

`print(a != b) # True`

Important

`=` # Assignment

`==` # Comparison

Example:

`x = 10`

Means: assign 10 to x.

`x == 10`

Means: check whether x is equal to 10.

Logical Operators

Logical operators are used to combine multiple conditions.

and

Both conditions must be True.

`age > 18 and age < 25`

or

At least one condition must be True.

`age > 25 or age == 20`

not

Reverses the result.

```
not(age == 20)
```

Assignment Operators

=

+=

-=

*=

/=

//=

%=

**=

Example:

```
x = 10
```

```
x += 5
```

This is the same as:

```
x = x + 5
```

So the final value of x is 15.

7. Conditional Statements

Conditional statements allow a program to **make decisions based on conditions**.

if

```
age = 20
```

```
if age >= 18:
```

```
    print("Adult")
```

if-else

```
age = 16
```

```
if age >= 18:
```

```
    print("Adult")
```

```
else:
```

```
    print("Minor")
```

if-elif-else

Used when there are multiple conditions.

```
marks = 75
```

```
if marks >= 90:
```

```
    print("A")
```

```
elif marks >= 60:
```

```
    print("B")
```

```
else:
```

```
    print("C")
```

Output:

B

Important

Python uses **indentation** to define a block.

```
if age >= 18:
```

```
    print("Adult")
```

8. Loops

Loops are used to **repeat a block of code**.

for Loop

A for loop is commonly used to iterate over a sequence or range.

```
for i in range(1, 6):
```



```
print(i)
```

Output:

1

2

3

4

5

range()

```
range(start, stop)
```

The start value is included, but the stop value is excluded.

```
range(2, 7)
```

Gives:

2 3 4 5 6

while Loop

A while loop runs as long as its condition is True.

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i += 1
```

Output:

1

2

3

4

5

Important

The condition of a while loop should eventually become False. Otherwise, an **infinite loop** can occur.

9. Strings

A string represents text.

```
name = "Arati"
```

Both single and double quotes can be used:

```
name = 'Arati'
```

```
name = "Arati"
```

Indexing

Python uses **zero-based indexing**.

```
A r a t i
```

```
0 1 2 3 4
```

```
name[0] # A
```

```
name[2] # a
```

```
name[4] # i
```

Negative Indexing

```
A r a t i
```

```
-5 -4 -3 -2 -1
```

```
name[-1] # i
```

```
name[-2] # t
```

Slicing

Slicing is used to extract a part of a string.

```
name[1:4]
```

Output:

```
rat
```

Rule

```
[start : stop]
```

Start is included, stop is excluded.

len()

Used to find the length of a string.

```
len("Arati")
```

Output:

5

Common String Methods

```
name.upper()
```

```
name.lower()
```

```
name.replace("A", "B")
```

```
name.strip()
```

```
name.startswith("A")
```

```
name.endswith("i")
```

Important

Strings are **immutable**.

This means their individual characters cannot be changed directly.

```
name = "Arati"
```

```
name[0] = "B"
```

This will produce an error.

10. Lists

A list stores multiple values in a single variable.

```
marks = [80, 90, 75, 88]
```

Lists are **mutable**, so their elements can be changed.

```
marks[1] = 95
```

Now:

```
[80, 95, 75, 88]
```

Indexing

```
marks[0]
```

```
marks[2]
```

Important List Methods

append()

Adds a value at the end.

```
marks.append(85)
```

insert()

Adds a value at a specific position.

```
marks.insert(1, 88)
```

remove()

Removes a specific value.

```
marks.remove(95)
```

pop()

Removes the last element by default.

```
marks.pop()
```

len()

Returns the length of the list.

```
len(marks)
```

List Slicing

```
numbers = [10, 20, 30, 40, 50]
```

```
numbers[1:4]
```

Output:

```
[20, 30, 40]
```

sort() vs sorted()

```
numbers.sort()
```

sort() changes the original list.

```
new_numbers = sorted(numbers)
```

sorted() returns a new sorted list.

11. Tuple

A tuple is similar to a list, but it is **immutable**.

```
numbers = (10, 20, 30)
```

Tuple supports indexing:

```
numbers[0] # 10
```

```
numbers[-1] # 30
```

But this is not allowed:

```
numbers[0] = 100
```

It produces an error.

Remember

List → Mutable

Tuple → Immutable

12. Set

A set stores **unique values**.

```
numbers = {1, 2, 2, 3, 3}
```

The result will contain:

```
{1, 2, 3}
```

Duplicate values are automatically removed.

Set Methods

```
numbers.add(4)
```

```
numbers.remove(2)
```

Important

Sets do not support normal indexing.

```
numbers[0]
```

This produces an error.

13. Dictionary

A dictionary stores data in **key-value pairs**.

```
student = {  
    "name": "Arati",  
    "age": 20,  
    "marks": 90  
}
```

Here:

"name" → Key

"Arati" → Value

Accessing a Value

```
student["age"]
```

Output:

20

Updating a Value

```
student["marks"] = 95
```

Adding a New Pair

```
student["city"] = "Pune"
```

Important Dictionary Methods

```
student.keys()
```

```
student.values()
```

```
student.items()
```

Dictionary Loop

```
for key, value in student.items():
```

```
    print(key, value)
```

14. Functions

A function is a **reusable block of code** designed to perform a specific task.

Defining a Function

```
def greet():  
    print("Hello Arati")
```

Calling a Function

```
greet()
```

Output:

Hello Arati

Parameters

Functions can accept input using parameters.

```
def greet(name):  
    print("Hello", name)
```

```
greet("Arati")
```

Here, name is a parameter.

return

return sends a value back from the function.

```
def add(a, b):  
    return a + b
```

```
result = add(10, 20)
```

```
print(result)
```

Output:

30

Important Difference

print() → displays output on the screen

return → sends a value back from the function

15. Modules and Packages

Module

A module is a **Python file** containing code such as functions, variables or classes.

Example:

```
import math
```

```
print(math.sqrt(16))
```

Output:

```
4.0
```

Different Import Styles

```
import math
```

```
from math import sqrt
```

```
print(sqrt(25))
```

Using an alias:

```
import math as m
```

```
print(m.sqrt(36))
```

Package

A package is a **collection of related Python modules**.

Remember

Module → One Python file

Package → Collection of modules

16. Exception Handling

Exception handling is used to **handle runtime errors** safely.

try-except

try:


```
num = int(input("Enter number: "))  
  
except ValueError:  
  
    print("Please enter a valid number")
```

If the user enters an invalid value, the exception can be handled instead of allowing the program to stop unexpectedly.

Example

```
try:  
  
    x = 10 / 0  
  
except ZeroDivisionError:  
  
    print("Cannot divide by zero")
```

Output:

Cannot divide by zero

finally

The finally block normally executes after the try/except process.

```
try:  
  
    print(10 / 2)  
  
except:  
  
    print("Error")  
  
finally:  
  
    print("Done")
```

Output:

5.0

Done

Remember

try → code that may cause an exception

except → handles the error

finally → executes at the end

17. File Handling

Python can be used to **read, write and append** data in files.

File Modes

"r" → Read

"w" → Write / Overwrite

"a" → Append

Reading a File

with open("data.txt", "r") as file:

```
    content = file.read()
```

```
    print(content)
```

Writing to a File

with open("data.txt", "w") as file:

```
    file.write("Hello Arati")
```

⚠ "w" can overwrite existing content.

Appending to a File

with open("data.txt", "a") as file:

```
    file.write("\nHello again")
```

"a" preserves the existing content and adds new data at the end.

with open()

Using with open() is preferred because Python automatically handles closing the file after the block is finished.